IMLAB · QUALITY ASSURANCE

Test Case Report

Detailed record of the test cases executed against the ImLab pathology platform — browser flows, API checks, database cross-checks and the Indian Standard Time (IST) remediation.

REPORT DATE	26/10/2026	VERSION	1.0
PREPARED FOR	Gaurav Karadbhajne	PREPARED BY	Qoder — AI QA / verification
TEST WINDOW	22/10/2026 – 26/10/2026 (IST)	ENVIRONMENT	Local dev · tenant CITYDIAG · MySQL 8
APPLICATION	ImLab · .NET 8 API + Angular 19	DATA SNAPSHOT	46 live bookings · booking 49 (lab 2610049)

66	58	4	4
TEST CASES CATALOGUED	LIVE-PASS	CODE-VERIFIED	PARTIAL

Result at a glance

58 cases passed with live browser and/or API evidence cross-checked against MySQL. 4 additional cases were verified in code only because they could not be reproduced with the available data or were not re-run after the final build. 4 cases were partially verified. **Five defects were found during the window; all five were fixed and re-verified. No open functional defect remains in the tested scope**, subject to the limitations in Section 6.

All times in this report are IST (Asia/Kolkata, UTC+05:30) unless marked UTC.

Dev credentials are deliberately omitted from this document.

Contents

How this report is organised

1 **Executive summary**
Result dashboard, coverage and defect overview

2 **Environment, scope and method**
Build under test, evidence tools, procedure

3 **Defect log**
5 defects — root cause, fix, verification

4 **Test execution sessions**
What was run, when, and with which evidence

5 **Test cases**
66 detailed cases across 10 modules

6 **Known limitations**
What was not exercised and why

7 **Appendix A — UTC → IST evidence**
Stored value vs displayed value, surface by surface

8 **Appendix B — Database evidence snapshot**
Counts and booking-49 money ledger

9 **Appendix C — Status definitions & sign-off**
How to read PASS / CODE / PARTIAL

Module index: 5.1 Authentication · 5.2 Registration · 5.3 Search & KPIs · 5.4 Billing & Payments · 5.5 Packages · 5.6 Delete Log & Restore · 5.7 Date & Time (IST) · 5.8 Printable Documents · 5.9 Dashboard · 5.10 API & Contract

1 Executive summary

This report catalogues every test case executed during the 22–26 October 2026 verification window, including the full IST time-zone suite that closed the last critical display defect. Each case records its test data, steps, expected result and the actual evidence observed.

Headline result

66 test cases: 58 live-pass, 4 code-verified, 4 partial. **5 defects found, 5 fixed, 0 open.** The most significant work in the window was the IST remediation: all timestamps are stored in UTC and now display in Indian Standard Time everywhere — grid, drawer, delete log, registration history, dashboard, printable receipts and PDFs — with IST day boundaries used for "today", tokens, date filters and daily summaries.

Coverage by module

MODULE	CASES	PASS	CODE-VERIFIED	PARTIAL	PRIMARY EVIDENCE
5.1 Authentication	2	1	1	0	Browser login; auth code path
5.2 Registration	8	5	1	2	Browser + API + MySQL
5.3 Search & KPIs	10	10	0	0	Browser + API + MySQL counts
5.4 Billing & Payments	8	7	1	0	Browser + API + receipts
5.5 Packages	7	6	1	0	Booking 49 + audit rows
5.6 Delete Log & Restore	9	9	0	0	Browser + API + MySQL
5.7 Date & Time (IST)	13	12	0	1	API JSON + browser + receipts
5.8 Printable Documents	3	2	0	1	HTML receipts + PDF HTTP
5.9 Dashboard	1	1	0	0	Browser + API
5.10 API & Contract	5	5	0	0	Live HTTP responses
Total	**66**	**58**	**4**	**4**	**22–26 Oct 2026**

Defects in the window

ID	DEFECT	SEVERITY	STATUS
DEF-001	Timestamps displayed as UTC clock instead of IST; server day math used UTC days	High	Fixed & verified
DEF-002	Delete-log filters silently ignored because of an MVC parameter-name collision	High	Fixed & verified
DEF-003	Package component delete/restore matched the wrong package identifier	Medium	Fixed & verified
DEF-004	Collection could exceed the bill total on collect / add-tests / registration	High	Fixed & verified

ID	DEFECT	SEVERITY	STATUS
DEF-005	Dashboard header date was browser-local, not IST	Low	Fixed & verified

How to read the status labels

PASS — exercised live in the browser and/or API and cross-checked where relevant against MySQL. **CODE-VERIFIED** — the behaviour is present and compiled, but the exact scenario was not re-run end-to-end with live data. **PARTIAL** — part of the scenario was verified live; the remaining part is code-verified or blocked by absent data. See Appendix C.

2 Environment, scope and method

The tests were executed against a freshly built copy of the application, not against a stale development server. Every result was cross-checked at the layer where it could fail: UI, API or database.

Environment under test

PRODUCT	ImLab — multi-tenant pathology laboratory management platform (rebuild of the legacy eazylab product).
APPLICATION STACK	.NET 8 API + MVC views, Entity Framework Core 8 + Pomelo, Angular 19 standalone client, QuestPDF for PDF receipts.
CLIENT BUNDLE	Angular production build, served from `src/ImLab.Web/wwwroot` by the tested API host.
API HOST	Temporary Kestrel instance started from `D:/ImLab/.buildcheck` at `http://localhost:5099` (Development). The VS-hosted API was stopped first to avoid locked output DLLs. The temporary host was stopped and the build folder removed after verification.
DATABASE	MySQL 8.0 at `127.0.0.1:3307`; master schema `imlab_master`; tenant schema `imlab_t_citydiag`.
TENANT / USER	Company ID `CITYDIAG`, username `admin` (dev-only password omitted from this report).
BROWSER	Chromium driven through Playwright MCP; assertions at 1440 px, 1000 px and 600 px viewport widths. Console checked at `error` level.
EVIDENCE TOOLS	Page DOM / rendered text, in-page API JSON, MySQL CLI queries, HTTP status / content-type / byte size for documents.
TEST WINDOW	22/10/2026 – 26/10/2026 (IST). The IST suite and the final package / search checks were executed on 26/10/2026.

Verification method

Build and serve

- Angular rebuilt so the production bundle contained the `istDate` pipe and filter changes.
- Backend built into `.buildcheck`, because the running Visual Studio API held the default output files.
- Temporary API restarted after every client rebuild so the static web-assets manifest matched the new chunk hashes.

Evidence captured

- Browser: row counts, exact displayed strings, KPI labels, status chips, empty states, responsive column counts.
- API: response status, JSON values and UTC suffix on timestamps.
- MySQL: independent counts and stored UTC values for the same scope.
- Documents: HTML date line and PDF HTTP response (status, type, size).

Data snapshot used for expected values

LIVE (NON-DELETED) BOOKINGS	**46** at the time of the final IST checks.
BOOKINGS ON 24/10/2026 (IST DAY)	**2** — labs 2610049 (05:23 PM IST) and 2610047 (01:14 PM IST).

BOOKINGS FROM 23/10/2026 ONWARD	**21**.
TEXT SEARCH "GAURAV" (ALL TIME)	**20**.
OUTSTANDING VISITS (ALL TIME)	**43**.
SPECIFIC CORPORATE FILTER USED IN THE TEST	**3**. (A raw count of rows with any non-empty corporate value is 8 — a different predicate, not a conflict.)
REBOOK VISITS	**14** rows carry `is_rebook = 1`.
DELETE LOG AT FINAL SNAPSHOT	**5 rows** — 3 bookings, 1 investigation, 1 payment.
BOOKING 49 / LAB 2610049	Total ₹350; gross collected ₹350; refunded ₹300; net collected ₹50; balance ₹300. Lines: ₹50 test, ₹300 package, two ₹0 component tests.

> **Important test-data note**
>
> During the "today default" proof, a disposable QA booking was temporarily moved into the 26/10/2026 IST window and then restored to its original UTC value `2026-10-24 11:53:44` (24/10/2026, 05:23 PM IST). Booking 49's published/refunded state reflects the money-guard tests and is dev test data, not production data.

3 Defect log

Five defects were identified during this verification window. All were fixed in place and re-verified with live evidence. No defect in this list remains open.

DEF-001 — **Timestamps displayed the UTC clock instead of IST; server day boundaries also used UTC days** — HIGH

Observed: a visit booked at 11:20 IST rendered as 05:50 — the browser was printing the raw UTC clock. Date-scoped features (today default, tokens, daily summary, delete-log day filter) computed "today" from UTC rather than the lab's Indian day.

Root cause: MySQL `DATETIME` values materialise with `DateTimeKind.Unspecified`; System.Text.Json emitted them with no zone marker, so the browser interpreted the UTC clock as local time. Server-side day math used UTC.

Fix: registered `UtcDateTimeJsonConverter` so every API `DateTime` is emitted with `Z`; introduced `IndianTime` for all lab-day math and conversions; added the `istDate` pipe as the single UI formatter; date filters send explicit `+05:30` day boundaries; Razor receipts and QuestPDF use `IndianTime.FromUtc`.

Verification: API returns `2026-10-24T11:53:44Z`; grid, drawer, delete log, dashboard and both HTML receipts show IST; 24 Oct filter returns exactly the 2 expected rows; today default still returns 0 of 46. **PASS — fixed and verified 26/10/2026.**

DEF-002 — **Delete-log filters were silently ignored** — HIGH

Observed: on Reports → Delete log, choosing Type, typing text, or setting From/To appeared to apply but the result set did not change — even the Type filter was dropped.

Root cause: the MVC action was declared as `Deleted([FromQuery] DeleteLogQuery q, …)` while the client sends a query key named `q` for text search. MVC bound the whole complex parameter from that single value and discarded every other filter.

Fix: renamed the action parameter to `filter` with an explanatory comment; the `q` query key remains the text term.

Verification: Type, text, Type + text and From–To all narrow the list; a 24 Oct filter returns 4 rows; paging probes match. **PASS — fixed and verified.**

DEF-003 — **Package component delete/restore matched the wrong package identifier** — MEDIUM

Observed: deleting a package removed the package charge but left its component tests mixed into the investigations list; restoring had the mirror problem.

Root cause: `BookingTest.PackageId` stores the *catalog* package id, while the delete/restore code compared it with the `booking_packages` row id.

Fix: component operations now use `p.PackageId == b.Id` (catalog id of the booking's package), and the audit detail records `components=N`.

Verification: on booking 49, deleting the ₹300 package moved the total ₹350 → ₹50, soft-deleted 2 components, and wrote audit `delete-package components=2`; restore returned the total to ₹350 and wrote `restore-package`. **PASS — fixed and verified 26/10/2026.**

DEF-004 **Money collected could exceed the bill total** HIGH

Observed: a user could collect more than the outstanding balance, overpay while adding tests, or register with a paid amount above the total.

Fix: server guards on every collection path (collect, add-tests, registration create) reject an overpay with a clear 400 ("…at most ₹X now"); the UI shows the same hint and disables Register / Save / Collect. Receipt delete and restore are also guarded so collected and refunded totals can never go negative.

Verification: over-collect, add-tests overpay and registration overpay all returned 400 with the expected message; valid partial/full collections still succeed. **PASS — fixed and verified 24/10/2026.**

DEF-005 **Dashboard header date was browser-local, not IST** LOW

Observed: the dashboard header used `new Date().toDateString()` and rendered a non-IST textual date; it changed with the browser locale instead of showing the lab day.

Fix: format weekday and date with `Intl.DateTimeFormat(…, { timeZone: 'Asia/Kolkata' })`; output is `Mon, 26/10/2026`.

Verification: dashboard header reads `Mon, 26/10/2026 · live overview`. **PASS — fixed and verified 26/10/2026.**

Defect trend

Three of the five defects were correctness or money-safety issues in the core front-desk workflow. All three had user-visible symptoms and were fixed at the root, not masked in the UI. The IST defect was cross-cutting and is the reason Section 5.7 exists as a dedicated suite.

4 Test execution sessions

The verification was incremental: each session added a module or closed a defect, and the regression surface was re-checked after every build.

DATE (IST)	FOCUS	EVIDENCE COLLECTED
22/10/2026	Registration rebuild and rebook flow — new patient, duplicate lookup, Available Records, same-UID rebook, billing, print option.	Browser E2E against the client; booking creation; patient UID continuity; receipt surface. Earlier UI-regression checks also covered the redesigned shell.
24/10/2026	Search page, Billing & Payments guards, Delete Log list/filter/restore, printable receipt/voucher, PDF generation.	Browser + API + MySQL. Receipts R26000022/23/24, deleted R26000025; refund and money-guard checks; delete-log restore; PDF 200 / 29,019 bytes.
26/10/2026	Package grouping/delete/restore, Search “today” default and KPI labels, full IST suite, dashboard header and console checks.	Browser at 1440/1000/600 px + API JSON + MySQL counts. Booking 49 package delete 350→50 and restore 350; 24 Oct = 2 rows; today = 0 of 46; delete log = 5 rows, 24 Oct filter = 4; conversion table captured.
26/10/2026	Report compilation (this document).	Evidence from the sessions above; no new product code was exercised for the report itself.

What “live-verified” means here

A PASS in this report means the exact assertion was observed in the running application during the stated session, and — where a count or money value was involved — independently reproduced in MySQL. Read-only code inspection alone is labelled CODE-VERIFIED. The report does not upgrade code inspection to a live pass.

5 Test cases

66 cases, grouped by module. Every card states the test data, the steps executed, the expected result and the actual evidence observed.

SECTION	MODULE	CASES	CASE IDS
5.1	Authentication	2	TC-AUTH-01 … 02
5.2	Registration	8	TC-REG-01 … 08
5.3	Search & KPIs	10	TC-SRCH-01 … 10
5.4	Billing & Payments	8	TC-MON-01 … 08
5.5	Packages	7	TC-PKG-01 … 07
5.6	Delete Log & Restore	9	TC-DL-01 … 09
5.7	Date & Time (IST)	13	TC-IST-01 … 13
5.8	Printable Documents	3	TC-PRN-01 … 03
5.9	Dashboard	1	TC-DASH-01
5.10	API & Contract	5	TC-API-01 … 05

5.1 Authentication

2 cases

Front-door access control and the failed-login policy: captcha after three wrong attempts, never a hard lockout.

1 PASS | **1** CODE-VERIFIED | **0** PARTIAL

TC-AUTH-01 — Valid login with Company ID, username and password — PASS

Layer: UI + API **Priority:** Critical **Executed:** 22–26 Oct 2026

TEST DATA / PRECONDITIONS	Tenant `CITYDIAG`, user `admin`, dev-only password; API and MySQL running; Angular bundle fresh.
STEPS	1. Open the login page. 2. Enter Company ID, username and password. 3. Submit and wait for the dashboard.
EXPECTED RESULT	HTTP 200; JWT stored; redirect to the dashboard; tenant-scoped data loads for `imlab_t_citydiag`.
ACTUAL RESULT & EVIDENCE	Used at the start of every browser session on 22, 24 and 26 Oct. Login succeeded, the dashboard rendered tenant data (46 live bookings; booking 49 lookup; delete log), and authenticated API calls returned 200. **PASS.**

TC-AUTH-02 — Third failed login requires a captcha — account is not locked — CODE-VERIFIED

Layer: UI + API **Priority:** High **Executed:** implementation review 16–26 Oct 2026

TEST DATA / PRECONDITIONS	A user with fewer than 3 failed attempts; captcha endpoint available at `GET /api/v1/auth/captcha`.
STEPS	1. Submit a wrong password three times. 2. Submit a fourth login without a captcha. 3. Request a captcha, solve the math question, and submit it with the correct password.
EXPECTED RESULT	After three failures the API returns 428 (Precondition Required) and the client shows the captcha widget; a correct answer proceeds; a wrong answer increments failures; the account is never hard-locked.
ACTUAL RESULT & EVIDENCE	Code path verified: `AuthService` requires a captcha when `FailedAttempts >= 3`; `CaptchaService` keeps a one-shot answer in `IMemoryCache`; lockout was removed and the login widget was delivered. A full end-to-end browser confirmation of the three-failure sequence was not recorded in the final session. **CODE-VERIFIED.**

5.2 Registration

8 cases

The daily front-desk workflow: new booking, numbering, validation, duplicate detection, same-UID rebook, billing and print.

5 PASS | **1** CODE-VERIFIED | **2** PARTIAL

TC-REG-01 — Full new-patient registration creates the booking and advances all number series — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 22 Oct 2026

TEST DATA / PRECONDITIONS	New patient; patient type WalkIn; age in years/months/days; referral doctor and sample collector; at least one department test; partial payment; print receipt selected.
STEPS	1. Open Patients → New registration. 2. Enter name, age Y/M/D, mobile, patient type, referral and delivery mode. 3. Pick tests from the department tabs, adjust a line charge, enter a partial payment and mode. 4. Submit Register.
EXPECTED RESULT	Booking saved; daily token computed; lab number 2610xxx; bill number B…; patient UID P26-…; billing split persisted; receipt available.
ACTUAL RESULT & EVIDENCE	Registration E2E completed successfully; a new patient and booking were created, the bill and token rendered, and all three number series advanced. By 24 Oct the series had reached lab **2610049**; 46 live bookings existed at the final snapshot. **PASS.**

TC-REG-02 — Mandatory-field validation blocks an incomplete registration — PASS

Layer: UI + API **Priority:** High **Executed:** 22 Oct 2026

TEST DATA / PRECONDITIONS	Registration form with one or more mandatory fields empty and no test selected.
STEPS	1. Clear the first name and mobile. 2. Leave the test cart empty. 3. Attempt to submit.
EXPECTED RESULT	Angular form invalid; field-level errors shown; submit blocked; a direct API call with invalid data returns 400 with validation messages.
ACTUAL RESULT & EVIDENCE	The submit path was blocked by the reactive form during the E2E run; the API's FluentValidation rules reject invalid booking payloads with field-level 400 responses. **PASS.**

TC-REG-03 — Existing patient triggers “Available Records” with View and Rebook actions — PASS

Layer: UI + API **Priority:** High **Executed:** 22 Oct 2026; re-checked 26 Oct 2026

TEST DATA / PRECONDITIONS	An existing patient with a past visit (e.g. the “Gaurav” test family; 20 matching bookings in the DB).
STEPS	1. Type the existing mobile or name into a new registration. 2. Wait for the lookup. 3. Inspect the Available Records popup.
EXPECTED RESULT	Popup lists Name, Patient ID, Mobile, Balance, Date and Action; View opens the prior booking; Rebook pre-fills a new registration.
ACTUAL RESULT & EVIDENCE	Popup opened with the expected columns and actions; the visit date rendered as `24/10/2026` (IST). Balance and patient ID were displayed. **PASS.**

TC-REG-04 — Rebook reuses the UID, creates a new visit, and marks the visit as rebooked — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 22 Oct 2026; DB cross-check 26 Oct 2026

TEST DATA / PRECONDITIONS	Existing patient with at least one prior visit.
STEPS	1. Use Rebook from the duplicate popup or the Search page. 2. Confirm the pre-filled patient details. 3. Add tests and save.
EXPECTED RESULT	Same patient UID; new lab number, bill and token; `is_rebook = 1` on the visit; REBOOK ribbon on the receipt and amber Rebook chip in Search.
ACTUAL RESULT & EVIDENCE	Same-UID rebook verified end-to-end; the new visit received fresh lab/bill/token numbers. At the final check **14 visits** carried `is_rebook = 1`, and the Search grid displayed the amber chip on the rebooked visit. **PASS.**

TC-REG-05 — Billing split recalculates live and overpayment is blocked — PASS

Layer: UI + API **Priority:** Critical **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A registration cart with a known total; payment entered above the total.
STEPS	1. Set lines so the total is known. 2. Apply a percentage or rupee discount and a remark. 3. Enter a paid amount greater than the total and try to Register.
EXPECTED RESULT	Total, discount and balance recalculate live; overpay is blocked on the client and by the API with a clear 400 “at most ₹X” message.
ACTUAL RESULT & EVIDENCE	Live totals and balance matched the line arithmetic; an overpay attempt was rejected by the server guard and the Register path remained blocked with the amount hint. **PASS.**

TC-REG-06 — Backdated booking is accepted at the IST clock time of entry

PARTIAL

Layer: UI + API + DB **Priority:** Medium **Executed:** 21–26 Oct 2026

TEST DATA / PRECONDITIONS	Front desk enters a booking for a past date late in the day.
STEPS	1. Pick a past booking date. 2. Complete the registration and save. 3. Inspect the stored UTC value and the displayed IST value.
EXPECTED RESULT	Past date accepted; the clock time remains the IST time of entry on that date; the visit appears under that date filter; future dates are rejected separately (TC-REG-07).
ACTUAL RESULT & EVIDENCE	The booking-date picker defaulted to `2026-10-26` (IST today) in the live form. The backdate conversion (`BookingDate` + `IndianTime.ToUtc` with the current IST time) is present and compiled. A dedicated backdated save was not recorded after the IST rebuild. **PARTIAL.**

TC-REG-07 — Future booking date is rejected

CODE-VERIFIED

Layer: API validation **Priority:** Medium **Executed:** code review 26 Oct 2026

TEST DATA / PRECONDITIONS	A booking payload with `BookingDate` set to tomorrow (IST).
STEPS	1. Submit a registration dated tomorrow. 2. Observe the validation response.
EXPECTED RESULT	400 validation error; no booking created.
ACTUAL RESULT & EVIDENCE	FluentValidation rule `LessThanOrEqualTo(IndianTime.Today)` is present and the solution compiles; the rule uses the lab's IST day. Not separately exercised live in the final session. **CODE-VERIFIED.**

TC-REG-08 — Registration print option opens a printable receipt

PARTIAL

Layer: UI + server-rendered document **Priority:** Medium **Executed:** 22 Oct 2026; receipt surface re-checked 26 Oct 2026

TEST DATA / PRECONDITIONS	A successful registration with the print-receipt option selected.
STEPS	1. Register with print receipt enabled. 2. Inspect the opened receipt.
EXPECTED RESULT	Receipt shows token, lab number, patient, tests, billing and the IST booking date; the print dialog can be invoked.
ACTUAL RESULT & EVIDENCE	The print-receipt option is present in the registration flow and the server-rendered receipt surface was verified separately (booking 49, TC-PRN-01). A dedicated registration-print click was not recorded in the final session. **PARTIAL.**

5.3 Search & KPIs

10 cases

Date-scoped default, filter behaviour, KPI consistency, the rebook indicator and responsive layout.

10 PASS | **0** CODE-VERIFIED | **0** PARTIAL

TC-SRCH-01 — No filters: Search opens on today's IST visits only — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	46 live bookings in the tenant; 0 bookings on the 26/10/2026 IST day.
STEPS	1. Open Patients → Search with no search text and no filters. 2. Inspect the grid, KPI cards and empty state.
EXPECTED RESULT	Only today's visits; empty state "No visits booked today"; all KPI cards 0; labels "Patients today" and "booked today".
ACTUAL RESULT & EVIDENCE	Live: 0 rows, Patients 0, Bookings 0, empty state shown. MySQL `today26_ist = 0` against `live_total = 46`. **PASS.**

TC-SRCH-02 — Date filter 24/10/2026 returns exactly the two visits for that IST day — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	From = To = 2026-10-24; two bookings in the tenant on that IST day.
STEPS	1. Set From and To to 24/10/2026. 2. Apply and inspect the grid and cards.
EXPECTED RESULT	2 rows; Patients 2 and Bookings 2; displayed times in IST; date note "24 Oct 2026".
ACTUAL RESULT & EVIDENCE	2 rows: lab 2610049 at `05:23 PM IST` (DB 11:53:44 UTC) and lab 2610047 at `01:14 PM IST` (DB 07:44:12 UTC). MySQL `d24_ist = 2`. **PASS.**

TC-SRCH-03 — From-date filter 23/10/2026 returns 21 visits — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	From = 2026-10-23, To empty; 21 live bookings from that IST day onward.
STEPS	1. Set From to 23/10/2026. 2. Apply; scroll the grid and compare the count with MySQL.
EXPECTED RESULT	21 rows; cards reflect the same scope.
ACTUAL RESULT & EVIDENCE	Grid returned 21 rows; MySQL `from23_ist = 21`. **PASS.**

TC-SRCH-04	**Text search “Gaurav” returns 20 bookings across all time**	PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Search term “Gaurav”; 20 matching live bookings in MySQL.
STEPS	1. Type “Gaurav” into the Search box. 2. Wait for the 300 ms debounce. 3. Compare the row count and the cards.
EXPECTED RESULT	20 rows; today default removed; cards scoped to the matching text.
ACTUAL RESULT & EVIDENCE	20 rows returned; MySQL `gaurav = 20`. Text search is debounced (300 ms) and resets the grid before reloading. **PASS.**

TC-SRCH-05	**Outstanding-only filter returns 43 visits**	PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Outstanding-only toggle; 43 visits with a non-zero balance in MySQL.
STEPS	1. Enable Outstanding only. 2. Apply; inspect the grid and the Outstanding card.
EXPECTED RESULT	43 rows; Outstanding card count 43 with the amount due; today default removed.
ACTUAL RESULT & EVIDENCE	43 rows; MySQL `outstanding = 43`. **PASS.**

TC-SRCH-06	**Specific corporate filter returns 3 visits**	PASS

Layer: UI + API + DB **Priority:** Medium **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A specific corporate value used by the test data.
STEPS	1. Choose the corporate value in the filter. 2. Apply and compare with a filtered MySQL count.
EXPECTED RESULT	Only that corporate’s 3 visits.
ACTUAL RESULT & EVIDENCE	3 rows returned, matching the specific-value query. Note: a raw count of all rows with *any* non-empty corporate value is 8; that is a different predicate and was not the assertion here. **PASS.**

TC-SRCH-07 — Patients KPI equals Bookings KPI in every scope, with scope-aware labels — PASS

Layer: UI + API **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	No filters; then a from-date filter; then a text search.
STEPS	1. Note the Patients and Bookings cards with no filters. 2. Apply a date filter and re-read both cards and the labels. 3. Apply a text search and re-read the note.
EXPECTED RESULT	Patients always equals Bookings (Patients counts visits — the user's explicit product choice); labels change from "Patients today / booked today" to "Patients / <range or matching filters>".
ACTUAL RESULT & EVIDENCE	0/0 today, 2/2 for 24 Oct, matching counts in the text-search scope; labels and notes updated correctly, including the date-range note "24 Oct 2026". **PASS.**

TC-SRCH-08 — Rebooked visits show the amber Rebook chip at visit level — PASS

Layer: UI + DB **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	14 visits with `is_rebook = 1`; at least one visible in the current filter.
STEPS	1. Apply a filter that includes a rebooked visit. 2. Inspect the row's status chips. 3. Confirm a non-rebook visit does not carry the chip.
EXPECTED RESULT	Amber "Rebook" chip on rebooked visits only, not on every visit of that patient.
ACTUAL RESULT & EVIDENCE	Chip rendered on the rebooked visit row and matched `is_rebook = 1`; the indicator is visit-level, not patient-level. **PASS.**

TC-SRCH-09 — "Today" default is a live IST-day query — proof by moving and restoring a booking — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Disposable QA booking 49; originally booked 2026-10-24 11:53:44 UTC.
STEPS	1. Move the booking's UTC timestamp into the 26/10/2026 IST window. 2. Refresh Search with no filters; observe the count. 3. Restore the original timestamp and refresh again.
EXPECTED RESULT	Default count changes 0 → 1 → 0 without touching any filter; the 24 Oct filter still finds the restored booking.
ACTUAL RESULT & EVIDENCE	Default showed 1 while the booking was in today's IST window, then 0 after restore to `2026-10-24 11:53:44`. Stats are cached for ~30 s, so KPI refreshes can lag a mutation by up to half a minute. **PASS.**

TC-SRCH-10	**KPI card grid is responsive at 1000 px and 600 px**	PASS

Layer: UI **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Search page with the six KPI cards.
STEPS	1. View at 1440 px, then 1000 px, then 600 px. 2. Inspect the card columns and table overflow.
EXPECTED RESULT	6 columns → 3 columns → 2 columns; labels remain readable; no page-level horizontal overflow.
ACTUAL RESULT & EVIDENCE	Column counts observed at 6 → 3 → 2 at the three widths; labels and values remained legible. **PASS.**

5.4 Billing & Payments

8 cases

Collect, refund, receipt life-cycle and the guards that keep collected money between ₹0 and the bill total.

7 PASS | **1** CODE-VERIFIED | **0** PARTIAL

TC-MON-01 Collect the full outstanding balance — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A booking with a positive balance and no collection for the amount.
STEPS	1. Open the visit and choose Collect. 2. Enter exactly the balance, pick a payment mode and submit.
EXPECTED RESULT	Balance becomes ₹0; visit becomes Fully paid; a payment receipt is created; KPIs refresh.
ACTUAL RESULT & EVIDENCE	Full collection succeeded, the receipt appeared in the drawer and the booking moved to Fully paid. Receipt R26000023 for ₹250 is part of the booking-49 ledger. **PASS.**

TC-MON-02 Collect a partial amount — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A booking with a balance greater than the amount to collect.
STEPS	1. Collect an amount smaller than the balance. 2. Inspect the balance, the Part paid card and the receipt.
EXPECTED RESULT	Balance reduced by the exact amount; visit Part paid; receipt created; totals consistent.
ACTUAL RESULT & EVIDENCE	Partial collection produced the expected balance and Part paid state; receipt R26000022 for ₹100 was created on booking 49. **PASS.**

TC-MON-03 Collect above the balance is rejected — PASS

Layer: UI + API **Priority:** Critical **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49: balance ₹300, gross ₹350, refunded ₹300, net ₹50.
STEPS	1. Enter a collect amount greater than ₹300 (e.g. ₹301, ₹1,000). 2. Attempt to submit.
EXPECTED RESULT	400 with "…at most ₹300 now"; UI shows the same hint and blocks submission; no receipt created.
ACTUAL RESULT & EVIDENCE	Over-collect was rejected with the expected amount guard; the UI hint matched and the Collect action was blocked. **PASS.**

TC-MON-04 — **Overpay is blocked when adding tests and during registration** — PASS

Layer: UI + API **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	An add-tests cart whose paid amount exceeds the new total; a registration payload with paid > total.
STEPS	1. Add tests, enter a paid amount above the new total and save. 2. Repeat with a new registration.
EXPECTED RESULT	Both paths return 400 with a clear balance hint; no receipt or booking is created.
ACTUAL RESULT & EVIDENCE	Add-tests overpay and registration overpay were both rejected by the server guard; the client showed the same hint and disabled the save path. **PASS.**

TC-MON-05 — **Refund reduces net collection and increases the balance** — PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49 with collected money available to refund.
STEPS	1. Choose Refund and enter a valid amount. 2. Submit; inspect the balance, net collected and the refund receipt.
EXPECTED RESULT	Net collected falls by the refund; balance rises; refund receipt is created; gross/refunded/net stay internally consistent.
ACTUAL RESULT & EVIDENCE	Refund R26000024 for ₹300 created; booking 49 ended at gross ₹350, refunded ₹300, net ₹50, balance ₹300. **PASS.**

TC-MON-06 — **Receipt delete is blocked if it would make collected negative** — PASS

Layer: API + UI **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A receipt whose deletion would push the booking's net collected below zero.
STEPS	1. Attempt to delete the receipt. 2. Inspect the response and the booking totals.
EXPECTED RESULT	400 with a clear explanation; receipt remains; totals unchanged.
ACTUAL RESULT & EVIDENCE	The guarded delete was rejected and the receipt remained; totals stayed consistent. **PASS.**

TC-MON-07 **Receipt restore is blocked if the refund would go negative** PASS

Layer: API + UI **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A soft-deleted receipt whose restoration would make refunded money exceed gross collected.
STEPS	1. Open Reports → Delete log. 2. Attempt to restore the receipt.
EXPECTED RESULT	400 with a clear explanation; the row stays deleted; totals unchanged.
ACTUAL RESULT & EVIDENCE	The guarded restore was rejected with a clear 400; the delete-log row remained and the ledger stayed balanced. **PASS.**

TC-MON-08 **Collect form resets pay and discount fields after a successful collection** CODE-VERIFIED

Layer: UI **Priority:** Low **Executed:** code review 26 Oct 2026

TEST DATA / PRECONDITIONS	A successful collection followed by reopening the Collect form.
STEPS	1. Collect an amount successfully. 2. Reopen the Collect form and inspect the amount and discount fields.
EXPECTED RESULT	Pay amount, discount amount, discount percent and discount remark are cleared; no stale value from the previous collection.
ACTUAL RESULT & EVIDENCE	The success handler in `booking-view.component.ts` resets all four fields and is present in the current bundle build. Not separately re-clicked after the final rebuild. **CODE-VERIFIED.**

5.5 Packages

7 cases

How package charges, their component tests, delete/restore and the add-tests picker behave.

6 PASS | **1** CODE-VERIFIED | **0** PARTIAL

TC-PKG-01 — Package charge groups its ₹0 component tests in the Investigations tab — PASS

Layer: UI + API **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49: ₹50 test, ₹300 package, two ₹0 component tests.
STEPS	1. Open booking 49 → Investigations tab. 2. Inspect the package row and the component rows beneath it.
EXPECTED RESULT	One package row carrying the ₹300 charge; component tests indented ("↳") with ₹0; the ₹50 test remains a normal row.
ACTUAL RESULT & EVIDENCE	Package row rendered with the two component rows indented (classes `pkgrow` / `kidrow`); bill total ₹350. Catalog `PackageId` is now carried on package rows so the grouping survives reload. **PASS.**

TC-PKG-02 — Component without a visible package row falls back to "Included · In package" — CODE-VERIFIED

Layer: UI fallback **Priority:** Low **Executed:** code review 26 Oct 2026

TEST DATA / PRECONDITIONS	A package component whose package charge row is missing or filtered out.
STEPS	1. Render a booking with an orphan component row. 2. Inspect its label.
EXPECTED RESULT	Component still visible as "Included · In package", not counted as a separate charge.
ACTUAL RESULT & EVIDENCE	Fallback label exists in the template and the grouping logic handles orphan components. Booking 49 has no orphan row, so this path was not triggered live. **CODE-VERIFIED.**

TC-PKG-03 — Delete-package modal previews the bill and requires a reason — PASS

Layer: UI **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49 with a ₹300 package and a ₹350 total.
STEPS	1. Click Delete package on the package row. 2. Inspect the preview, then try to confirm with fewer than 3 characters.
EXPECTED RESULT	Modal shows the package and the bill total / balance after deletion; confirmation disabled until a reason of at least 3 characters is entered.
ACTUAL RESULT & EVIDENCE	Modal displayed the post-delete preview and enforced the reason rule; the confirm action stayed disabled until a valid reason. **PASS.**

TC-PKG-04	**Deleting a package soft-deletes its components and updates the bill**	PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49: total ₹350, package ₹300 with 2 components, plus a ₹50 test.
STEPS	1. Delete the package with a valid reason. 2. Inspect the Investigations tab, the bill and the audit trail.
EXPECTED RESULT	Package and its two components soft-deleted; total ₹350 → ₹50; the ₹50 test remains; audit entry `delete-package` with `components=2`.
ACTUAL RESULT & EVIDENCE	Total moved ₹350 → ₹50, both components disappeared, and the audit recorded `delete-package components=2`. **PASS.**

TC-PKG-05	**Restoring a package restores its components and the bill total**	PASS

Layer: UI + API + DB **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	The package deleted in TC-PKG-04, visible in Reports → Delete log.
STEPS	1. Open Reports → Delete log. 2. Restore the package row. 3. Reopen booking 49.
EXPECTED RESULT	Package and components active again; total back to ₹350; delete-log row resolved; audit `restore-package`.
ACTUAL RESULT & EVIDENCE	Restore returned the total to ₹350 with both components visible and wrote `restore-package` to the audit. **PASS.**

TC-PKG-06	**Component rows cannot be deleted on their own**	PASS

Layer: UI + API guard **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	A ₹0 component row inside an active package charge.
STEPS	1. Inspect the row actions on a component. 2. Attempt a direct component delete.
EXPECTED RESULT	No direct delete action on component rows; the API guard rejects a lone component delete; the supported action is Delete package.
ACTUAL RESULT & EVIDENCE	Component rows expose no delete action and the backend guard rejects a lone component delete; the package row is the supported deletion point. **PASS.**

TC-PKG-07	**Add-tests package picker and cart line list the included tests**	PASS

Layer: UI **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Add tests → Packages tab; a package with known component tests.
STEPS	1. Open the Packages tab in Add tests. 2. Inspect the package entry and add it to the cart.
EXPECTED RESULT	Package entry names its included tests; the cart line repeats the included test names; the package is charged once.
ACTUAL RESULT & EVIDENCE	Picker and cart line listed the included test names via `packageTestNames`; the package appeared as a single charge. **PASS.**

5.6 Delete Log & Restore

9 cases

Soft-deleted bookings, investigations, packages and receipts: list, filter, page and restore — with guards.

9 PASS | **0** CODE-VERIFIED | **0** PARTIAL

TC-DL-01 Delete log lists all supported soft-deleted types, newest first — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Final snapshot: 3 deleted bookings, 1 investigation, 1 payment.
STEPS	1. Open Reports → Delete log. 2. Inspect the types, order, reason and deleted-at column.
EXPECTED RESULT	All four supported kinds represented; newest first; reason and IST deletion time shown.
ACTUAL RESULT & EVIDENCE	5 rows displayed (3 bookings, 1 investigation, 1 payment) newest first, with IST times such as `26/10/2026, 10:30 AM IST` and `24/10/2026, 05:26 PM IST`. **PASS.**

TC-DL-02 Type filter narrows the delete log correctly — PASS

Layer: UI + API **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Delete log with more than one type present.
STEPS	1. Select Bookings, then Investigation, then Payment. 2. Compare row counts with MySQL.
EXPECTED RESULT	Only the selected type is listed; counts match the database.
ACTUAL RESULT & EVIDENCE	Type filter applied correctly for each kind; counts matched MySQL (3 bookings / 1 investigation / 1 payment). **PASS.**

TC-DL-03 Text filter is debounced and matches log content — PASS

Layer: UI + API **Priority:** Medium **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	Delete log rows with identifiable text (patient, test, receipt).
STEPS	1. Type a term into the text filter. 2. Wait for the 350 ms debounce; clear the field.
EXPECTED RESULT	List narrows to matching rows without a manual Apply; clearing restores the unfiltered list.
ACTUAL RESULT & EVIDENCE	Text filter narrowed the list after the debounce and clearing restored the full list. **PASS.**

TC-DL-04 — From–To date filter uses IST days — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Deletion timestamps on 24/10/2026 and 26/10/2026 (IST).
STEPS	1. Set From = To = 24/10/2026. 2. Apply and compare with the full list.
EXPECTED RESULT	Only rows deleted on that IST day (4 of the 5 final rows); the 26 Oct row is excluded.
ACTUAL RESULT & EVIDENCE	4 rows returned for 24 Oct; the 26 Oct payment was excluded; the day boundaries are converted from IST to UTC before querying. **PASS.**

TC-DL-05 — Type and text filters combine (intersection) — PASS

Layer: UI + API **Priority:** Medium **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	A type with at least two rows and a text term that matches only one of them.
STEPS	1. Select a type and type the term. 2. Verify the result is the intersection, not the union.
EXPECTED RESULT	Only rows satisfying both filters; clearing one restores the broader set.
ACTUAL RESULT & EVIDENCE	Type + text produced the expected intersection; this case specifically guards against the DEF-002 silent-drop regression. **PASS.**

TC-DL-06 — Paging returns stable pages and an empty tail — PASS

Layer: API **Priority:** Medium **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	4 delete-log rows at the time of the paging probe.
STEPS	1. Request `size=2` at offset 0. 2. Request offset 2. 3. Request offset 4.
EXPECTED RESULT	2 rows with `hasMore=true`; next 2 rows (last page); offset 4 returns an empty list with `hasMore=false`.
ACTUAL RESULT & EVIDENCE	Probes matched exactly: 2 rows hasMore, final 2 rows, then empty. (The final snapshot has 5 rows, so offset 4 would now return the fifth row — the probe evidence above is from the 4-row state.) **PASS.**

TC-DL-07	**Restore a deleted payment receipt and a deleted refund receipt**	PASS

Layer: UI + API + DB **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	Soft-deleted payment and refund receipts visible in the delete log.
STEPS	1. Restore the payment receipt. 2. Restore the refund receipt. 3. Inspect the booking's receipt drawer and totals.
EXPECTED RESULT	Both receipts reappear in the booking; their totals re-enter the ledger; invalid restores return a clear 400.
ACTUAL RESULT & EVIDENCE	Payment and refund restores succeeded and the receipts returned to the drawer with correct totals; guarded endpoints rejected invalid restores with 400. **PASS.**

TC-DL-08	**Restore a deleted package together with its component tests**	PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	The package deleted in TC-PKG-04 (booking 49).
STEPS	1. Open Reports → Delete log. 2. Restore the package row. 3. Inspect booking 49's Investigations tab and audit.
EXPECTED RESULT	Package and both components active; total ₹350; audit `restore-package`; log row resolved.
ACTUAL RESULT & EVIDENCE	Restore brought back the package and its components, returned the total to ₹350, and wrote the expected audit entry. **PASS.**

TC-DL-09	**Regression: all delete-log filters survive the MVC binding fix**	PASS

Layer: API + UI **Priority:** High **Executed:** 24 Oct 2026

TEST DATA / PRECONDITIONS	Delete-log page after renaming the action parameter from `q` to `filter` (DEF-002).
STEPS	1. Apply Type only. 2. Add text. 3. Add From–To. 4. Confirm each change alters the result set.
EXPECTED RESULT	Every filter is honoured, individually and combined; text still travels as the `q` query key.
ACTUAL RESULT & EVIDENCE	All filter combinations changed the list as expected; paging and restore continued to work. **PASS.**

5.7 Date & Time (IST)

13 cases

The cross-cutting convention: UTC storage, IST display, IST day boundaries — verified on every surface that shows a timestamp.

12 PASS | **0** CODE-VERIFIED | **1** PARTIAL

TC-IST-01 API emits stored timestamps with an explicit UTC marker — PASS

Layer: API **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49 stored as `2026-10-24 11:53:44` in MySQL (UTC).
STEPS	1. Fetch the booking through the API. 2. Inspect `bookedAt` and receipt `paidAt` values.
EXPECTED RESULT	JSON string ends in `Z`, e.g. `2026-10-24T11:53:44Z`; no offset-less timestamp is emitted.
ACTUAL RESULT & EVIDENCE	API returned `"bookedAt": "2026-10-24T11:53:44Z"`; receipt times were also zone-marked. **PASS.**

TC-IST-02 The UTC converter is registered globally for MVC JSON — PASS

Layer: API configuration **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	`Program.cs` MVC JSON options; a freshly built API.
STEPS	1. Inspect the JSON options registration. 2. Call multiple endpoints that return `DateTime` values.
EXPECTED RESULT	`UtcDateTimeJsonConverter` is added once to the MVC serializer; every endpoint routed through MVC JSON inherits it.
ACTUAL RESULT & EVIDENCE	Converter registered in `AddControllersWithViews(...).AddJsonOptions(...)`; bookings, receipts, delete log and dashboard payloads all carried zone-marked times; solution built clean. **PASS.**

TC-IST-03 Search grid shows IST times for stored UTC values — PASS

Layer: UI **Priority:** Critical **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	24 Oct filter; DB values 11:53:44 and 07:44:12 UTC.
STEPS	1. Apply the 24 Oct date filter. 2. Read the visit-time column.
EXPECTED RESULT	`24/10/2026, 05:23 PM IST` and `24/10/2026, 01:14 PM IST` — not the UTC clock.
ACTUAL RESULT & EVIDENCE	Both rows rendered exactly as expected via the `istDate` pipe at `Asia/Kolkata`. **PASS.**

TC-IST-04 — Booking drawer receipt list shows IST times — PASS

Layer: UI **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Receipts R26000022, R26000023 and refund R26000024 on booking 49.
STEPS	1. Open the booking drawer. 2. Inspect each receipt row's time.
EXPECTED RESULT	R22 05:23 PM, R23 05:27 PM, R24 05:30 PM — all IST, all in the drawer.
ACTUAL RESULT & EVIDENCE	Drawer showed `05:23 PM`, `05:27 PM` and refund `05:30 PM IST`, matching DB 11:53:44, 11:57:26 and 12:00:17 UTC. **PASS.**

TC-IST-05 — Delete-log timestamps are displayed in IST — PASS

Layer: UI **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Deletion timestamps from 24 Oct and 26 Oct; stored in UTC.
STEPS	1. Open Reports → Delete log. 2. Read the deleted-at column for all rows.
EXPECTED RESULT	IST times corresponding to the stored UTC values, e.g. 05:26 PM, 03:02 PM, 11:38 AM, 11:37 AM, and 26/10/2026 10:30 AM.
ACTUAL RESULT & EVIDENCE	All five rows matched the UTC→IST conversion table in Appendix A. **PASS.**

TC-IST-06 — Delete-log date filter uses IST day boundaries — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	5 delete-log rows; 4 deleted on 24 Oct IST and 1 on 26 Oct IST.
STEPS	1. Set From = To = 24/10/2026. 2. Apply and compare with the full list.
EXPECTED RESULT	4 rows for 24 Oct; the 26 Oct row excluded; no UTC-day bleed at the edges.
ACTUAL RESULT & EVIDENCE	4 rows returned; the 26 Oct payment excluded. Server converts the IST day to UTC before comparing `deleted_at`. **PASS.**

TC-IST-07	**Search date filter sends explicit IST boundaries**	PASS

Layer: UI + API **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	From/To date pickers on Search.
STEPS	1. Pick 24/10/2026 for From and To. 2. Inspect the outbound query. 3. Verify the returned rows.
EXPECTED RESULT	Query carries `T00:00:00+05:30` and `T23:59:59.9999999+05:30`; 2 expected rows returned.
ACTUAL RESULT & EVIDENCE	Outbound boundaries carried `+05:30`; API returned the 2 rows whose UTC instants fall inside the IST day. **PASS.**

TC-IST-08	**Registration booking-date default is the IST today**	PASS

Layer: UI **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Registration page opened on 26/10/2026 IST.
STEPS	1. Open Patients → New registration. 2. Read the booking-date field.
EXPECTED RESULT	`2026-10-26` (the lab's IST day), independent of the browser timezone.
ACTUAL RESULT & EVIDENCE	Field showed `2026-10-26`; the helper formats "today" with `Asia/Kolkata`. **PASS.**

TC-IST-09	**Available Records visit dates use IST calendar days**	PASS

Layer: UI **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	A prior visit stored as 2026-10-24 11:53:44 UTC.
STEPS	1. Trigger the duplicate/available-records popup. 2. Read the Date column.
EXPECTED RESULT	`24/10/2026`; no time-zone shift into 23 or 25 October.
ACTUAL RESULT & EVIDENCE	Popup showed `24/10/2026` via `istDate:'date'`. **PASS.**

TC-IST-10 Dashboard header shows the IST day — PASS

Layer: UI **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Dashboard loaded on 26/10/2026 IST.
STEPS	1. Open the dashboard. 2. Read the header date.
EXPECTED RESULT	`Mon, 26/10/2026`, with “live overview” and no locale-dependent text date.
ACTUAL RESULT & EVIDENCE	Header read `Mon, 26/10/2026 · live overview`. **PASS.**

TC-IST-11 “Today” figures on Dashboard and Search use the same IST day — PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	0 bookings on 26/10/2026 IST; 46 live bookings overall.
STEPS	1. Read the dashboard’s today registrations and revenue. 2. Open Search with no filters and read the cards. 3. Compare with a MySQL IST-day query.
EXPECTED RESULT	Dashboard today registrations 0 and revenue ₹0; Search Patients/Bookings 0; MySQL today count 0 — all agree.
ACTUAL RESULT & EVIDENCE	Dashboard showed 0 registrations and ₹0 revenue; Search showed 0/0 with the empty state; MySQL `today26_ist = 0`. **PASS.**

TC-IST-12 Outsource date column formats the service date in IST — PARTIAL

Layer: UI **Priority:** Low **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Outsource page; 0 rows in the tenant at the time of the check.
STEPS	1. Open the Outsource page. 2. Inspect the date column binding.
EXPECTED RESULT	Service date rendered as `dd/MM/yyyy` via `istDate:'date'`.
ACTUAL RESULT & EVIDENCE	Page loaded; template uses `{{ r.bookedAt | istDate:'date' }}` and the build is clean. With 0 outsource rows there was no rendered date to compare against the database. **PARTIAL.**

TC-IST-13	**Search and Dashboard load without console errors after the IST rollout**	PASS

Layer: UI runtime **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Fresh production bundle; browser console at error level.
STEPS	1. Open Search; apply a date filter. 2. Open the Dashboard. 3. Read the console messages.
EXPECTED RESULT	No console errors or warnings related to the new pipe, converter or date filters.
ACTUAL RESULT & EVIDENCE	No error-level console messages on Search or Dashboard; both pages rendered the correct IST strings. **PASS.**

5.8 Printable Documents

3 cases

Server-rendered Razor receipts and vouchers, plus the QuestPDF download path.

2 PASS | **0** CODE-VERIFIED | **1** PARTIAL

TC-PRN-01 Booking receipt HTML shows the IST booking date — PASS

Layer: Server-rendered Razor **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49 / lab 2610049, booked 2026-10-24 11:53:44 UTC.
STEPS	1. Open the booking receipt HTML. 2. Read the date line.
EXPECTED RESULT	`24/10/2026, 05:23 PM IST`, not the UTC clock.
ACTUAL RESULT & EVIDENCE	Receipt rendered `Date 24/10/2026, 05:23 PM IST` via `IndianTime.FromUtc(Model.BookedAt)`. **PASS.**

TC-PRN-02 Payment voucher HTML shows the IST payment time — PASS

Layer: Server-rendered Razor **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Payment receipt R26000022 (booking 49), paid 2026-10-24 11:53:44 UTC.
STEPS	1. Open the payment voucher HTML. 2. Read the date line.
EXPECTED RESULT	`24/10/2026, 05:23 PM IST`.
ACTUAL RESULT & EVIDENCE	Voucher rendered `Date 24/10/2026, 05:23 PM IST` via `IndianTime.FromUtc(Model.PaidAt)`. **PASS.**

TC-PRN-03 Receipt PDF downloads successfully with the same IST date — PARTIAL

Layer: QuestPDF download **Priority:** Medium **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking receipt and payment-voucher PDF endpoints for booking 49 / payment 22.
STEPS	1. Request the receipt PDF. 2. Check status, content type and byte size. 3. Inspect the date-format code path.
EXPECTED RESULT	HTTP 200, PDF content type, non-trivial file; the date uses `IndianTime.FromUtc(...) + " IST"`.
ACTUAL RESULT & EVIDENCE	PDF returned HTTP 200, **29,019 bytes**; QuestPDF date span uses `IndianTime.FromUtc(...).ToString("dd/MM/yyyy, hh:mm tt") + " IST"`. QuestPDF compresses its content streams, so the rendered date text could not be machine-extracted from the file in this environment; the format is code-verified and generation is live-verified. **PARTIAL.**

5.9 Dashboard

1 case

Landing page load, KPI values and the IST day shown in the header.

1 PASS | **0** CODE-VERIFIED | **0** PARTIAL

TC-DASH-01	**Dashboard loads tenant KPIs with IST-scoped “today” values**	PASS

Layer: UI + API + DB **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Tenant CITYDIAG; 0 bookings on 26/10/2026 IST; 46 live bookings overall.
STEPS	1. Open the dashboard after login. 2. Read the KPI cards, the header date and the revenue panel. 3. Check the console.
EXPECTED RESULT	Dashboard renders; today registrations 0; revenue today ₹0; header shows `Mon, 26/10/2026`; no console errors.
ACTUAL RESULT & EVIDENCE	Dashboard loaded with 0 registrations, ₹0 revenue and the IST header; API `/api/v1/dashboard` returned 200; no console errors. **PASS.**

5.10 API & Contract

5 cases

Endpoint-level checks that back the UI assertions: status, shape, counts and document responses.

5 PASS | **0** CODE-VERIFIED | **0** PARTIAL

TC-API-01 Login endpoint returns a usable token — PASS

Layer: API **Priority:** Critical **Executed:** 22–26 Oct 2026

TEST DATA / PRECONDITIONS	Dev tenant and admin user; valid credentials.
STEPS	1. POST `/api/v1/auth/login`. 2. Use the returned token on an authenticated endpoint.
EXPECTED RESULT	HTTP 200; token and user/company data returned; subsequent calls authorised.
ACTUAL RESULT & EVIDENCE	Login returned 200 and the token authorised bookings, receipts, delete-log and dashboard calls throughout the sessions. **PASS.**

TC-API-02 Bookings list returns the expected counts for each filter scope — PASS

Layer: API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	46 live bookings; the filter scopes listed in Section 2.
STEPS	1. Call `/api/v1/bookings` with the 24 Oct IST range. 2. Repeat with From 23 Oct, text “Gaurav” and outstanding-only.
EXPECTED RESULT	2, 21, 20 and 43 rows respectively; no filter returns today’s 0.
ACTUAL RESULT & EVIDENCE	All API counts matched the UI and MySQL: 2 / 21 / 20 / 43; unfiltered request returned 0 for 26 Oct. **PASS.**

TC-API-03 Booking detail returns the correct money ledger and lines — PASS

Layer: API + DB **Priority:** High **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Booking 49 (lab 2610049), post-refund state.
STEPS	1. GET the booking detail. 2. Compare total, gross, refunded, net, balance and line items with MySQL.
EXPECTED RESULT	Total ₹350; gross ₹350; refunded ₹300; net ₹50; balance ₹300; ₹50 test, ₹300 package, two ₹0 components.
ACTUAL RESULT & EVIDENCE	API detail matched the DB ledger and line composition exactly. **PASS.**

TC-API-04	**Receipt HTML and PDF endpoints respond correctly**	PASS

Layer: API **Priority:** High **Executed:** 26 Oct 2026

TEST DATA / PRECONDITIONS	Booking receipt 49 and payment voucher 22 endpoints.
STEPS	1. Request the HTML receipt. 2. Request the PDF receipt. 3. Check status, content type and size.
EXPECTED RESULT	HTML 200 with the IST date; PDF 200, PDF content type, non-trivial size.
ACTUAL RESULT & EVIDENCE	HTML receipts rendered with `24/10/2026, 05:23 PM IST`; PDF returned 200 and 29,019 bytes. **PASS.**

TC-API-05	**Dashboard and delete-log endpoints serve the tested scopes**	PASS

Layer: API **Priority:** Medium **Executed:** 24–26 Oct 2026

TEST DATA / PRECONDITIONS	Authenticated session; 5 delete-log rows; dashboard tenant data.
STEPS	1. GET `/api/v1/dashboard`. 2. GET `/api/v1/bookings/deleted` with no filter and with the 24 Oct range.
EXPECTED RESULT	Dashboard 200 with today/7-day figures; delete-log 200 with 5 rows unfiltered and 4 for 24 Oct; filters honoured.
ACTUAL RESULT & EVIDENCE	Both endpoints returned 200 with the expected counts and scopes; no filter silently dropped. **PASS.**

6 Known limitations

These points bound the claims in this report. They are not necessarily defects; they are the parts of the product or test surface that were not fully exercised.

REF	LIMITATION	IMPACT / NEXT ACTION
L1	The Outsource page had no rows in the dev database, so its service-date column could not be compared with a real stored value.	Code-verified only. Create one outsource record and re-check the displayed date.
L2	The registration flow's print-receipt click was not separately recorded after the final IST rebuild; the receipt surface itself was verified via booking 49.	Re-run one registration with Print receipt checked and confirm the opened document.
L3	The generated PDF uses compressed content streams; the rendered date text could not be extracted in this environment.	PDF generation and the IST format code path are verified. A visual check of the downloaded PDF is advisable before handover.
L4	The three-failure captcha sequence was not re-run end-to-end in the final session.	Code-verified. Perform one manual failed-login sequence before calling the auth flow fully accepted.
L5	The collect-form reset is present in the success handler but was not re-clicked after the final rebuild.	Code-verified. One collect-and-reopen cycle will close the gap.
L6	Search KPI statistics are cached for about 30 seconds.	A mutation can take up to 30 s to appear in the cards; grid rows reload immediately. This is a performance choice, not a defect.
L7	"Corporate = 3" is the result of a specific corporate-value filter. Raw rows with any non-empty corporate value number 8.	Different SQL predicates; both were observed and are not contradictory. Do not compare the two numbers as if they were the same query.
L8	Database timestamps remain UTC by design; only the display and day-boundary math are IST.	Direct SQL reports will show UTC. Use IST conversion for any report written by hand.
L9	The final checks ran against a temporary API host on port 5099, not the user's Visual Studio-hosted API.	After stopping debug, rebuild and restart, perform a Ctrl+F5 smoke check on :5000 (and the :4200 dev server if used).
L10	These flows are verified by agent/manual execution, not by a repeatable automated regression suite.	The 3 starter unit tests pass, but there is no end-to-end test project for registration, search, money or delete-log. That remains a follow-up.
L11	Dev test data remains in the tenant (booking 49, soft-deleted rows, rebook visits).	Do not seed production from this database. Clean test data before handover/demo if required.
L12	Earlier UI checks sometimes ran against the Angular dev server on :4200; the final IST suite ran against the production bundle served by the API.	The production bundle is the authoritative target for the IST claims in this report.

Net position

> Within the tested scope, no functional defect is open. The remaining gaps are evidence gaps (code-verified or partial), not known failures. Closing L1–L5 would convert all 8 non-live cases into live passes.

7 Appendix A — UTC → IST evidence

The same stored value, seen at every surface. IST = UTC + 05:30. All rows below were captured during the final session.

STORED VALUE (UTC)	DISPLAYED VALUE (IST)	SURFACE / EVIDENCE
`2026-10-24 11:53:44`	**24/10/2026, 05:23 PM IST**	Search grid (lab 2610049), drawer receipt R26000022, booking receipt, payment voucher
`2026-10-24 07:44:12`	**24/10/2026, 01:14 PM IST**	Search grid (lab 2610047) on the 24 Oct filter
`2026-10-24 11:57:26`	**24/10/2026, 05:27 PM IST**	Drawer receipt R26000023 (₹250)
`2026-10-24 12:00:17`	**24/10/2026, 05:30 PM IST**	Drawer refund receipt R26000024 (₹300)
`2026-10-26 05:00:50`	**26/10/2026, 10:30 AM IST**	Delete log — payment R26000025 (₹200)
`2026-10-24 11:56:07`	**24/10/2026, 05:26 PM IST**	Delete log — Pus culture investigation
`2026-10-24 09:32:20`	**24/10/2026, 03:02 PM IST**	Delete log — booking
`2026-10-24 06:08:09`	**24/10/2026, 11:38 AM IST**	Delete log — booking
`2026-10-24 06:07:19`	**24/10/2026, 11:37 AM IST**	Delete log — booking
`API: 2026-10-24T11:53:44Z`	**24/10/2026, 05:23 PM IST**	JSON carries an explicit `Z`; client converts with `Asia/Kolkata`

IST day boundaries in UTC

- 24/10/2026 IST = `2026-10-23 18:30:00` UTC → `2026-10-24 18:30:00` UTC.
- 26/10/2026 IST = `2026-10-25 18:30:00` UTC → `2026-10-26 18:30:00` UTC.
- Search filters send `T00:00:00+05:30` / `T23:59:59.9999999+05:30`; the API compares UTC instants.

Why the defect happened

- MySQL `DATETIME` has no zone; EF/Pomelo materialised it as `Unspecified`.
- JSON without a zone marker was parsed by the browser as local time, so a UTC clock was printed.
- Fix: always emit `Z` from the API; always format with `Asia/Kolkata` in the UI; do all lab-day math with `IndianTime`.

8 Appendix B — Database evidence snapshot

Independent MySQL counts used as the expected values, and the money ledger for the primary test booking.

Counts at the final snapshot

SCOPE	PREDICATE SUMMARY	ROWS
Live bookings	Not soft-deleted	**46**
24/10/2026 (IST day)	IST day converted to UTC bounds	**2**
From 23/10/2026 (IST)	Lower bound at IST midnight	**21**
Today 26/10/2026 (IST)	IST day converted to UTC bounds	**0**
Text “Gaurav”	Name/mobile search, all time	**20**
Outstanding	Balance > 0, all time	**43**
Specific corporate filter	Exact corporate value used in the test	**3**
Any non-empty corporate	Different predicate (reference only)	**8**
Rebook visits	`is_rebook = 1`	**14**
Delete log — bookings	Soft-deleted bookings	**3**
Delete log — investigations	Soft-deleted investigations	**1**
Delete log — payments	Soft-deleted payment receipts	**1**
Delete log total	All supported kinds	**5**

Booking 49 / lab 2610049 — money ledger

BILL TOTAL	**₹350.00**
GROSS COLLECTED	₹350.00
REFUNDED	₹300.00
NET COLLECTED	₹50.00
BALANCE	₹300.00
LINE COMPOSITION	₹50 standalone test; ₹300 package charge; two ₹0 component tests
RECEIPTS	R26000022 ₹100 payment; R26000023 ₹250 payment; R26000024 ₹300 refund; R26000025 ₹200 payment deleted (delete log)
BOOKING TIME	2026-10-24 11:53:44 UTC → 24/10/2026, 05:23 PM IST

PACKAGE DELETE/RESTORE	Delete: total ₹350 → ₹50, audit `delete-package components=2`. Restore: total ₹50 → ₹350, audit `restore-package`

> **Data provenance**
>
> All values were read directly from MySQL 8 on port 3307 (`imlab_t_citydiag`) and compared with the API and UI. Booking 49 is disposable QA data and its values may change if further tests are run. The report's statuses refer to the state observed on 26/10/2026.

9 Appendix C — Status definitions & sign-off

How the labels in Section 5 should be interpreted, and the closing position for this verification window.

LABEL	DEFINITION
PASS	The assertion was exercised live in the running application and observed as expected. Count and money assertions were independently cross-checked in MySQL. A PASS is not inferred from code inspection.
CODE-VERIFIED	The implementation is present, compiled and inspected, but the exact scenario was not exercised end-to-end with live data in this window. This label is deliberately conservative.
PARTIAL	Part of the scenario was live-verified; the remaining part is code-verified or could not be exercised because the required data was absent (e.g. zero rows, compressed PDF text).

Closing position

Test cases catalogued	**66**
Live-pass	**58**
Code-verified	**4**
Partial	**4**
Defects found	**5**
Defects fixed & re-verified	**5**
Open functional defects in tested scope	**0**

Recommended next actions

1. Close L1–L5 with one short live pass: create an outsource row, register with print receipt, download and visually open a PDF, run the three-failure captcha flow, and reopen the collect form after a successful collection.
2. After restarting the user's Visual Studio debug host, rebuild and Ctrl+F5 once to confirm the same behaviour on :5000.
3. Start a small automated end-to-end suite for registration, money guards, search scoping and delete-log filters; today these checks are repeatable manually but not by CI.
4. Keep the UTC-in-database / IST-on-display rule: any new timestamp surface must use `IndianTime` or the `istDate` pipe, never `DateTime.Now` or a raw locale format.

Report sign-off

Prepared by: Qoder — AI QA / verification. **Prepared for:** Gaurav Karadbhajne. **Date:** 26/10/2026. **Scope:** ImLab local dev, tenant CITYDIAG, 22–26 October 2026. **Evidence sources:** Playwright-driven browser sessions, live API JSON, MySQL 8 queries, HTTP document responses. **Document version:** 1.0.

This document records executed verification and its evidence. It is not a substitute for a repeatable automated regression suite. Dev-only credentials are intentionally omitted.